Republic of the Philippines

University of Cabuyao

(PAMANTASAN NG CABUYAO)

COLLEGE OF COMPUTING STUDIES

Katapatan Mutual Homes, Brgy. Banay-banay, City of Cabuyao, Laguna 4025

FINALS HANDS-ON EXERCISE 2
ITEW2 CLIENT/ SERVER-SIDE SCRIPTING

Group Leader (LN, FN MI.):	1	JOCSON, BILLY JOHN B.		**Score:**	
Members (LN, FN MI.): In alphabetical Order	2	LUMPAY, JOHN ALEKS C.			
	3	MALANA, PATRICK KRU S.			
	4	MANDAP, VINCENT T.			
	5	NAÑOZ, LEANN JAE C.			
Section:		**Instructor:**	Prof. Janus Raymond C. Tan	**Sem./ A.Y.**	2nd/ 25-26
Project Link:	*https://drive.google.com/drive/folders/1sTuoeAGVhOywKuwK7EXKrvMzvobXU74r?usp=sharing*				
YouTube Link:	***https://youtu.be/YD-UC5noinw***				

CASE SCENARIO:

A local restaurant plans to develop a simple web-based Restaurant Order and Billing System as a CRUD (Create, Read, Update, Delete) application that enables staff to encode, view, update, and delete customer orders without using a database. The system will collect essential input data through an HTML form, including the customer name, menu item name, number of servings, and price per serving, which serve as the primary input factors. Using these inputs, the system must compute several billing values: the subtotal is calculated by multiplying the number of servings by the price per serving; a predefined service charge rate of 8% is applied to the subtotal; and a discount rate is determined based on predefined thresholds, where a 10% discount is applied if the subtotal is greater than or equal to 1000, a 5% discount is applied if the subtotal is at least 500 but less than 1000, and no discount is applied otherwise. The system will then compute the discount amount by multiplying the subtotal by the applicable discount rate and determine the final total bill using the formula: total equals subtotal plus service charge minus discount.

The application will be implemented using object-oriented programming (OOP), where a MenuItem class manages item-related data and computes the subtotal, and an Order class handles customer information while performing all billing-related computations, demonstrating a composition relationship between classes. Each transaction will be stored as an object inside an array of objects, and this array will be maintained using PHP sessions to ensure data persistence during the user's session. All stored records—including both input and computed factors such as subtotal, service charge, discount, and total—will be dynamically displayed in an HTML table using PHP. Additionally, each row in the table must provide Update (Edit) and Delete operations, allowing users to modify existing records or remove them entirely, thereby completing the full CRUD functionality within a single-file PHP application.

TEST CASES:

Customer	Item	Servings	Price
John	Burger	2	150
Ana	Pizza	3	200
Mark	Steak	2	600

ASSESSMENT:

Students will be assessed according to the submitted program code that satisfies the given program requirements. Score will be based on the following rubric:

Criteria	Weight	5 - Excellent	4 - Good	3 - Satisfactory	2 - Needs Improvement	1 - Poor
Video Presentation	5	Clear, engaging, well-paced; explains code & logic clearly; excellent visuals/audio.	Clear and understandable; good explanation with minor issues.	Adequate explanation; some unclear parts or minor distractions.	Hard to follow; poor pacing; lacks clarity in explanation.	Very unclear or incomplete; difficult to understand.
Functionality	3	All required features implemented and fully functional with no errors.	Most features implemented and mostly functional; minor bugs.	Basic features work; some bugs or missing minor parts.	Many features incomplete or non-functional.	Program does not work or missing major features.
Code Organization	2	Code is clean, well-structured, uses meaningful names, and has helpful comments.	Code mostly organized with some unclear naming or minimal comments.	Code is somewhat organized but hard to follow at times; few comments.	Poorly organized code; difficult to follow; lacks comments.	Code is messy, confusing, or very poorly structured.
Usability	2	User interface is intuitive and easy to navigate; clear prompts and outputs.	Mostly user-friendly; minor UI issues or unclear prompts.	Usability is average; some confusing or inconsistent elements.	Difficult to use; confusing menus or poor instructions.	User interface is very confusing or unusable.
Efficiency	1	Code uses loops and conditionals effectively; no redundant or unnecessary code.	Generally efficient; minor redundancy or inefficiency.	Moderate efficiency; some redundant or suboptimal code.	Inefficient code; excessive redundancy; slow or wasteful.	Very inefficient code; poor logic leading to issues.
Originality	2	Code is entirely original; demonstrates creativity and unique approach; no signs of plagiarism.	Code shows mostly original work with minor borrowed ideas but well-adapted.	Some originality evident; several elements follow standard or common solutions.	Limited originality; heavily relies on existing or template code.	Code is copied or plagiarized; no evidence of original contribution.

ACTUAL PROGRAM CODE:

MenuItem.php

```php
<?php
class MenuItem
{
  private $itemName;
  private $numberOfServings;
  private $pricePerServing;
  public function __construct($itemName, $numberOfServings, $pricePerServing)
  {
    $this->itemName = $itemName;
    $this->numberOfServings = $numberOfServings;
    $this->pricePerServing = $pricePerServing;
  }
  public function getSubtotal()
  {
    return $this->numberOfServings * $this->pricePerServing;
  }
  public function getItemName()
  {
    return $this->itemName;
  }
  public function getServings()
  {
    return $this->numberOfServings;
  }
  public function getPrice()
  {
    return $this->pricePerServing;
  }
}
?>
```

Order.php

```php
<?php
class Order
{
  private $customerName;
  private $menuItems = [];
  public function __construct($customerName)
  {
    $this->customerName = $customerName;
  }
  public function addItem($itemName, $servings, $price)
  {
    $this->menuItems[] = new MenuItem($itemName, $servings, $price);
  }
  public function getSubtotal()
```

```
    {
        $subtotal = 0;
        foreach ($this->menuItems as $item) {
            $subtotal += $item->getSubtotal();
        }
        return $subtotal;
    }
    public function getDiscountRate()
    {
        $subTotal = $this->getSubtotal();
        return ($subTotal >= 1000) ? 0.10 : (($subTotal >= 500) ? 0.05 : 0);
    }
    public function getDiscountAmount()
    {
        return $this->getSubtotal() * $this->getDiscountRate();
    }
    public function getServiceCharge()
    {
        return $this->getSubtotal() * 0.08;
    }
    public function getTotal()
    {
        return $this->getSubtotal() + $this->getServiceCharge() - $this->getDiscountAmount();
    }
    public function getItems()
    {
        return $this->menuItems;
    }
    public function getCustomerName()
    {
        return $this->customerName;
    }
}
?>
```

Index.php

```
<?php
include 'MenuItem.php';
include 'Order.php';
session_start();

if (!isset($_SESSION['orders'])) {
    $_SESSION['orders'] = [];
}

if (isset($_GET['delete'])) {
    $id = $_GET['delete'];
    unset($_SESSION['orders'][$id]);
    $_SESSION['orders'] = array_values($_SESSION['orders']);
    header("Location: " . $_SERVER['PHP_SELF']);
    exit;
}

$editData = null;
$editIndex = null;
if (isset($_GET['edit'])) {
    $editIndex = $_GET['edit'];
    if (isset($_SESSION['orders'][$editIndex])) {
        $obj = $_SESSION['orders'][$editIndex];
        $item = $obj->getItems()[0];
        $editData = [
            'name' => $obj->getCustomerName(),
            'item' => $item->getItemName(),
            'qty' => $item->getServings(),
            'prc' => $item->getPrice()
        ];
    }
}

if ($_SERVER['REQUEST_METHOD'] === "POST" && isset($_POST['submit'])) {
    $order = new Order($_POST['customerName']);
    $order->addItem($_POST['itemName'], $_POST['servings'], $_POST['price']);
    if (isset($_POST['update_index']) && $_POST['update_index'] !== "") {
        $_SESSION['orders'][$_POST['update_index']] = $order;
```

```
  } else {
    $_SESSION['orders'][] = $order;
  }
  header("Location: " . $_SERVER['PHP_SELF']);
  exit;
}
?>

<!DOCTYPE html>
<html>

<head>
  <title>Restaurant CRUD</title>
  <style>
    body {
      font-family: sans-serif;
      margin: 20px;
    }

    form {
      margin-bottom: 20px;
    }

    table {
      width: 100%;
      border-collapse: collapse;
    }

    th,
    td {
      border: 1px solid #ccc;
      padding: 8px;
      text-align: left;
    }
  </style>
</head>

<body>

  <h2><?= $editData ? "Edit Order" : "New Order" ?></h2>
  <form action="" method="POST">
    <input type="hidden" name="update_index" value="<?= $editIndex ?>">
    <input type="text" name="customerName" placeholder="Customer" value="<?= $editData['name'] ?? '' ?>" required>
    <input type="text" name="itemName" placeholder="Item" value="<?= $editData['item'] ?? '' ?>" required>
    <input type="number" name="servings" placeholder="Qty" value="<?= $editData['qty'] ?? '' ?>" required>
    <input type="number" step="0.01" name="price" placeholder="Price" value="<?= $editData['prc'] ?? '' ?>"
      required>
    <input type="submit" name="submit" value="<?= $editData ? 'Update' : 'Add' ?>">
    <?php if ($editData): ?><a href="<?= $_SERVER['PHP_SELF'] ?>">Cancel</a><?php endif; ?>
  </form>

  <table>
    <thead>
      <tr>
        <th>Customer</th>
        <th>Item (Qty)</th>
        <th>Subtotal</th>
        <th>Service (8%)</th>
        <th>Discount</th>
        <th>Total</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody>
      <?php
      $i = 0;
      $count = count($_SESSION['orders']);
      if ($count == 0) {
        echo "<tr><td colspan='7'>No records found.</td></tr>";
      } else {
        while ($i < $count) {
          $order = $_SESSION['orders'][$i];
          $item = $order->getItems()[0];
          ?>
          <tr data-id="<?= $i ?>">
```

```
            <td><?= $order->getCustomerName() ?></td>
            <td><?= $item->getItemName() ?> (x<?= $item->getServings() ?>)</td>
            <td><?= number_format($order->getSubtotal(), 2) ?></td>
            <td><?= number_format($order->getServiceCharge(), 2) ?></td>
            <td><?= number_format($order->getDiscountAmount(), 2) ?></td>
            <td><strong><?= number_format($order->getTotal(), 2) ?></strong></td>
            <td>
              <a href="?edit=<?= $i ?>">Edit</a> |
              <a href="?delete=<?= $i ?>" onclick="return confirm('Delete?')">Delete</a>
            </td>
          </tr>
          <?php
          $i++;
        }
      }
      ?>
    </tbody>
  </table>
</body>

</html>
```

SAMPLE OUTPUT WITH DESCRIPTION:

This appears if the user has not added any values yet:

New Order

Customer name | Item name | Number of servings | Price per serving | Add

Customer	Item (Qty)	Subtotal	Service (8%)	Discount	Total	Actions
No records found.						

Added all the details based on the provided test cases:

New Order

Customer name | Item name | Number of servings | Price per serving | Add

Customer	Item (Qty)	Subtotal	Service (8%)	Discount	Total	Actions
John	Burger (x2)	300.00	24.00	0.00	**324.00**	Edit | Delete
Ana	Pizza (x3)	600.00	48.00	30.00	**618.00**	Edit | Delete
Mark	Steak (x2)	1,200.00	96.00	120.00	**1,176.00**	Edit | Delete

If the user tries to edit, all the order details of that specific customer appears on the input fields. In this case I clicked the edit button on the row of John.

Edit Order

John | Burger | 2 | 150 | Update | Cancel

Customer	Item (Qty)	Subtotal	Service (8%)	Discount	Total	Actions
John	Burger (x2)	300.00	24.00	0.00	**324.00**	Edit | Delete
Ana	Pizza (x3)	600.00	48.00	30.00	**618.00**	Edit | Delete
Mark	Steak (x2)	1,200.00	96.00	120.00	**1,176.00**	Edit | Delete

Edit Order

John | Burger | 5 | 200 | Update | Cancel

Customer	Item (Qty)	Subtotal	Service (8%)	Discount	Total	Actions
John	Burger (x2)	300.00	24.00	0.00	**324.00**	Edit | Delete
Ana	Pizza (x3)	600.00	48.00	30.00	**618.00**	Edit | Delete
Mark	Steak (x2)	1,200.00	96.00	120.00	**1,176.00**	Edit | Delete

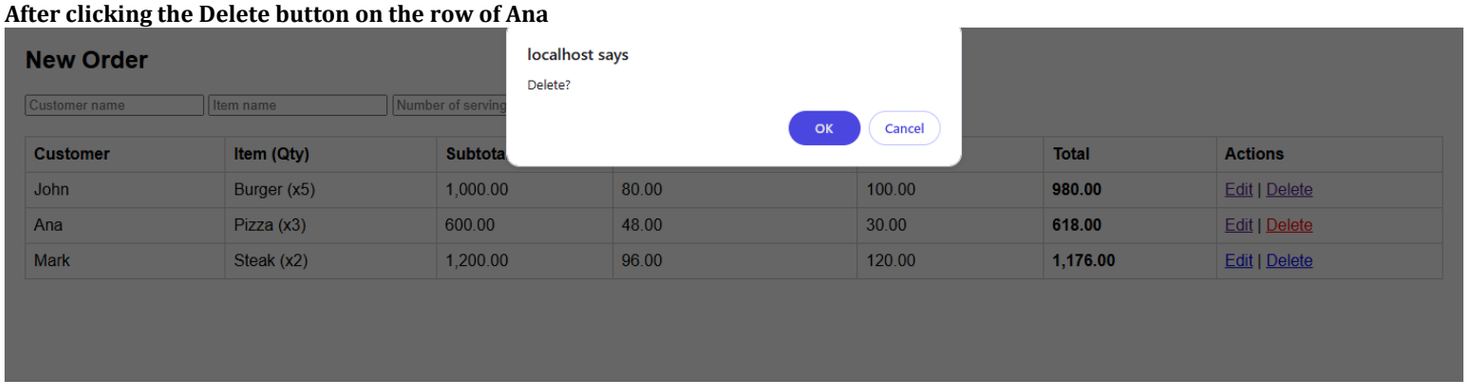
After clicking Update button:

New Order

Customer name | Item name | Number of servings | Price per serving | Add

Customer	Item (Qty)	Subtotal	Service (8%)	Discount	Total	Actions
John	Burger (x5)	1,000.00	80.00	100.00	**980.00**	Edit | Delete
Ana	Pizza (x3)	600.00	48.00	30.00	**618.00**	Edit | Delete
Mark	Steak (x2)	1,200.00	96.00	120.00	**1,176.00**	Edit | Delete

After clicking the Delete button on the row of Ana



After clicking ok, the row of Ana was successfully deleted:

New Order

Customer name | Item name | Number of servings | Price per serving | Add

Customer	Item (Qty)	Subtotal	Service (8%)	Discount	Total	Actions
John	Burger (x5)	1,000.00	80.00	100.00	**980.00**	Edit | Delete
Mark	Steak (x2)	1,200.00	96.00	120.00	**1,176.00**	Edit | Delete